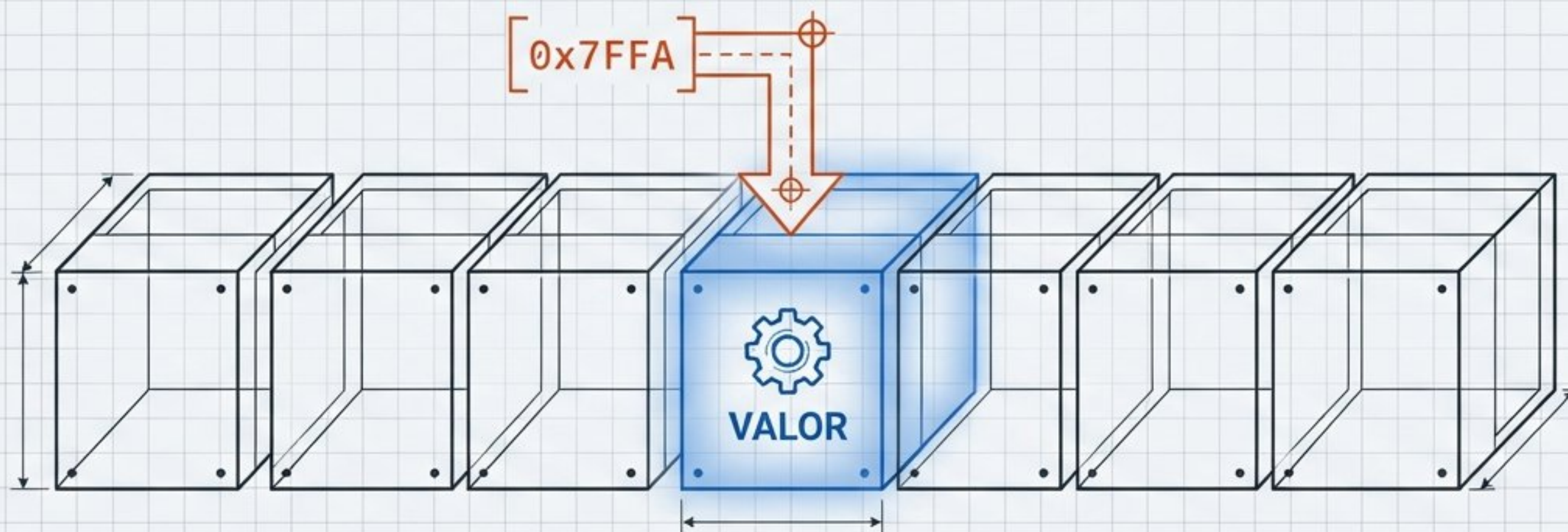
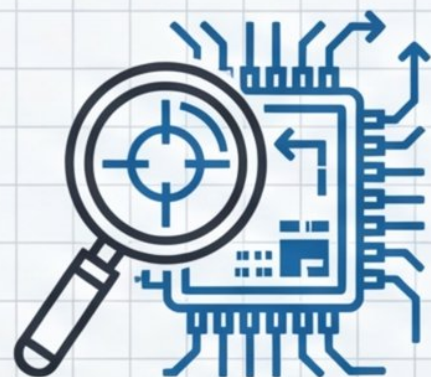


Semana 6: Ponteiros e Integração com Structs

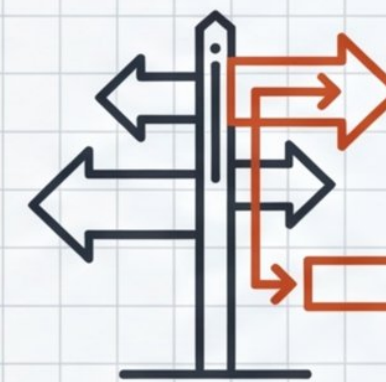
Teoria e Prática de Memória em C



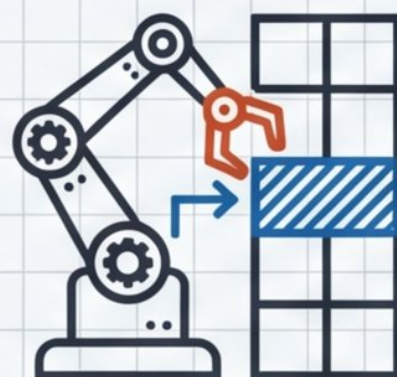
O Mapa da Aula: Objetivos da Semana



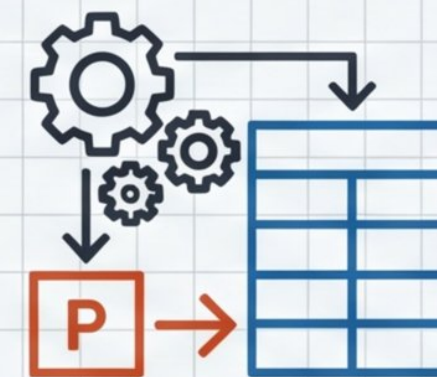
Compreender o conceito de endereço físico de memória.



Declarar e utilizar ponteiros para tipos básicos (int, float).

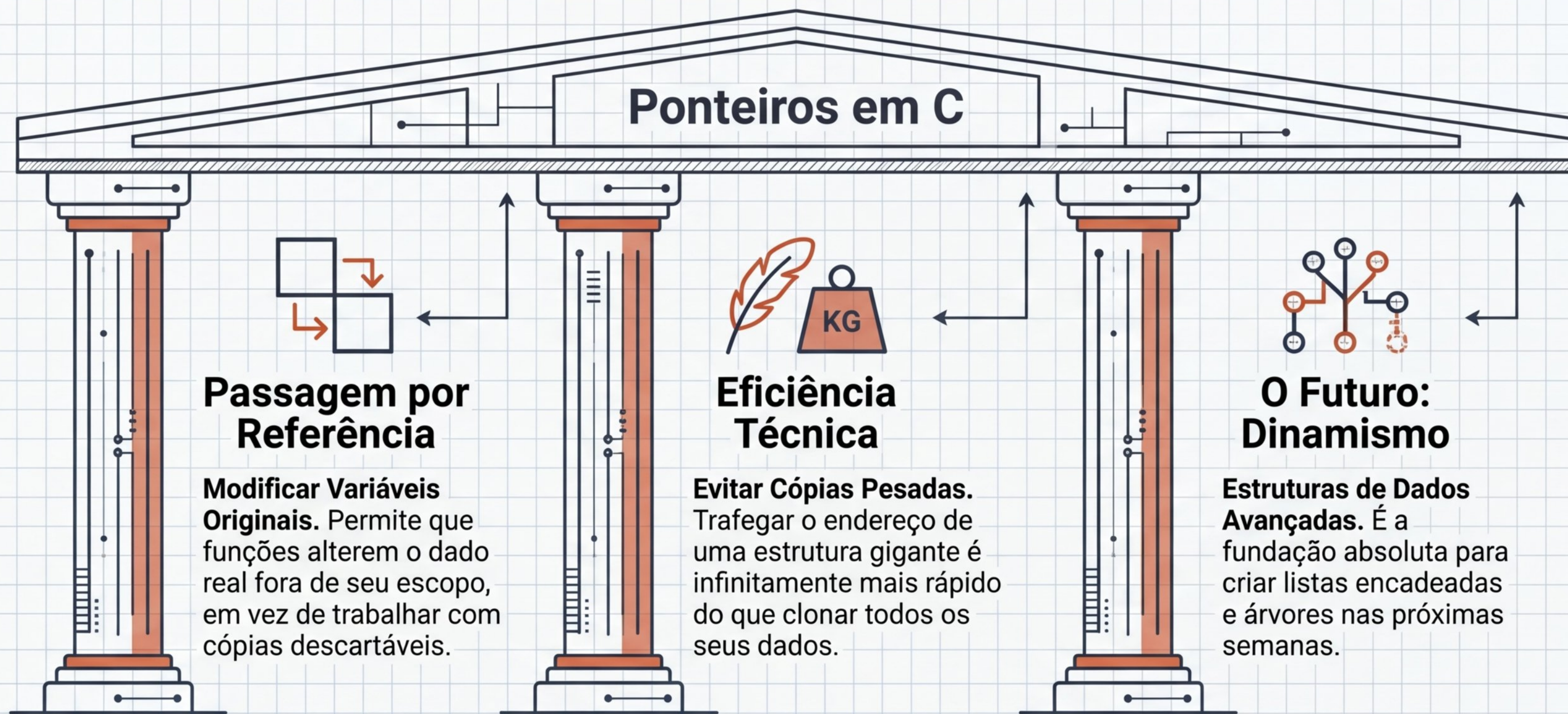


Acessar e modificar variáveis indiretamente via ponteiros.

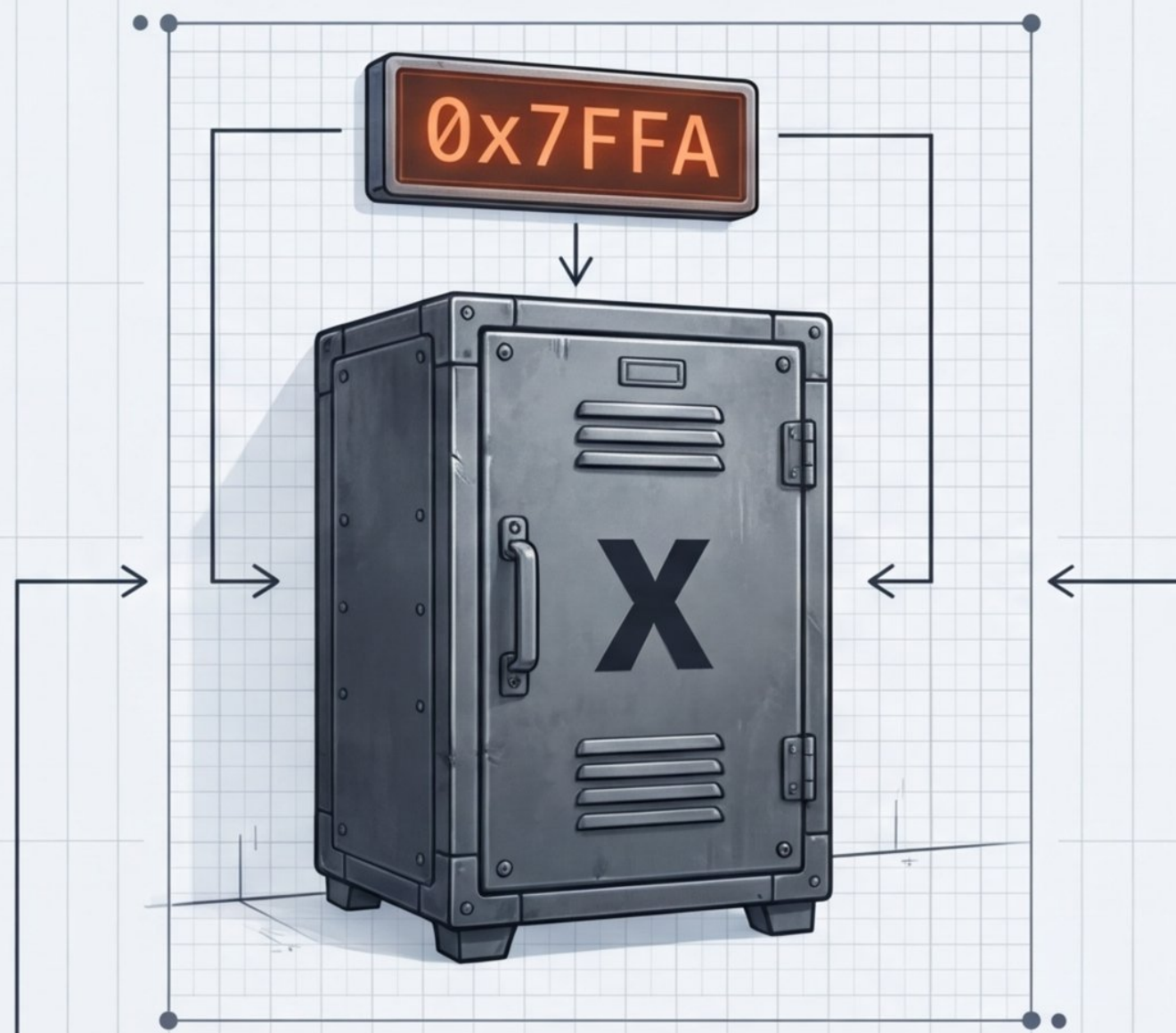


Utilizar ponteiros com estruturas (struct) através do operador `->`.

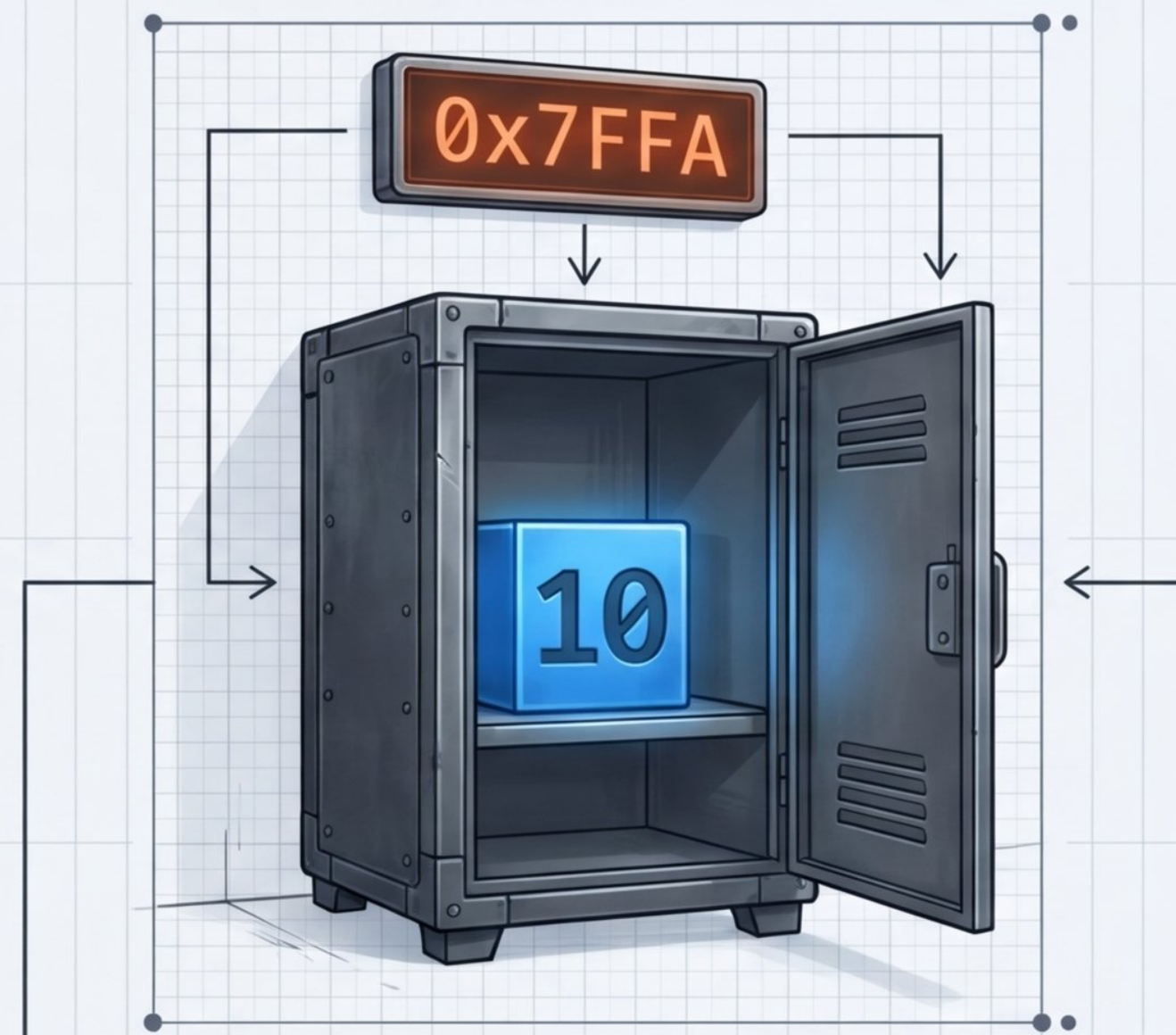
O Paradigma da Memória: Por que precisamos de ponteiros?



O Que é um Endereço de Memória?



O Endereço Físico (O "Onde").
Cada variável ocupa um espaço real no hardware. O sistema o identifica por um código hexadecimal.



O Valor Armazenado (O "O quê").
O conteúdo guardado dentro daquele espaço específico.

A Anatomia de uma Declaração

O Tipo Alvo.

Indica que este ponteiro guardará o endereço de um espaço que contém um número inteiro.

O Nome da Variável.

A nomenclatura escolhida para o seu ponteiro.

int *p;

O Indicador de Ponteiro.

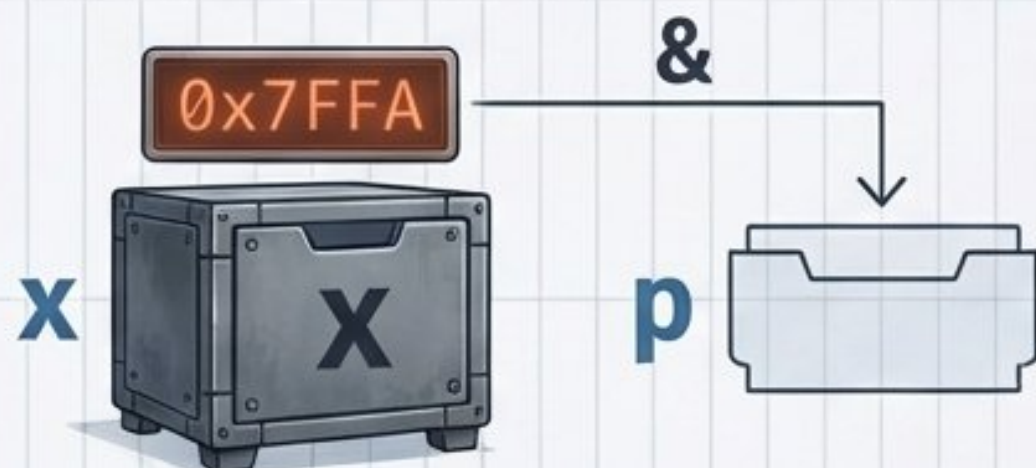
Na declaração, o asterisco o asterisco avisa ao compilador: "Esta variável não guardará um valor comum, guardará um endereço".

O Dueto Fundamental: Referência vs. Indireção



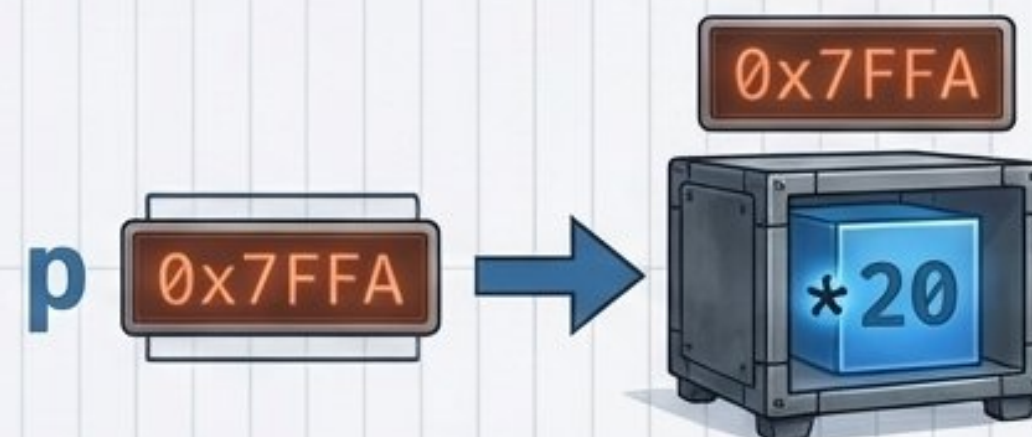
Operador de Endereço (&)

- **Pergunta:** "Onde você está guardado?"
- **Ação:** Extrai a localização física.
- **Exemplo:** `p = &x;` (O ponteiro **p** recebe o endereço físico da variável **x**).



Operador de Indireção (*)

- **Pergunta:** "O que tem aí dentro?"
- **Ação:** Acessa o conteúdo do endereço apontado.
- **Exemplo:** `*p = 20;` (Vai até o endereço guardado em **p** e altera o valor lá dentro para 20).



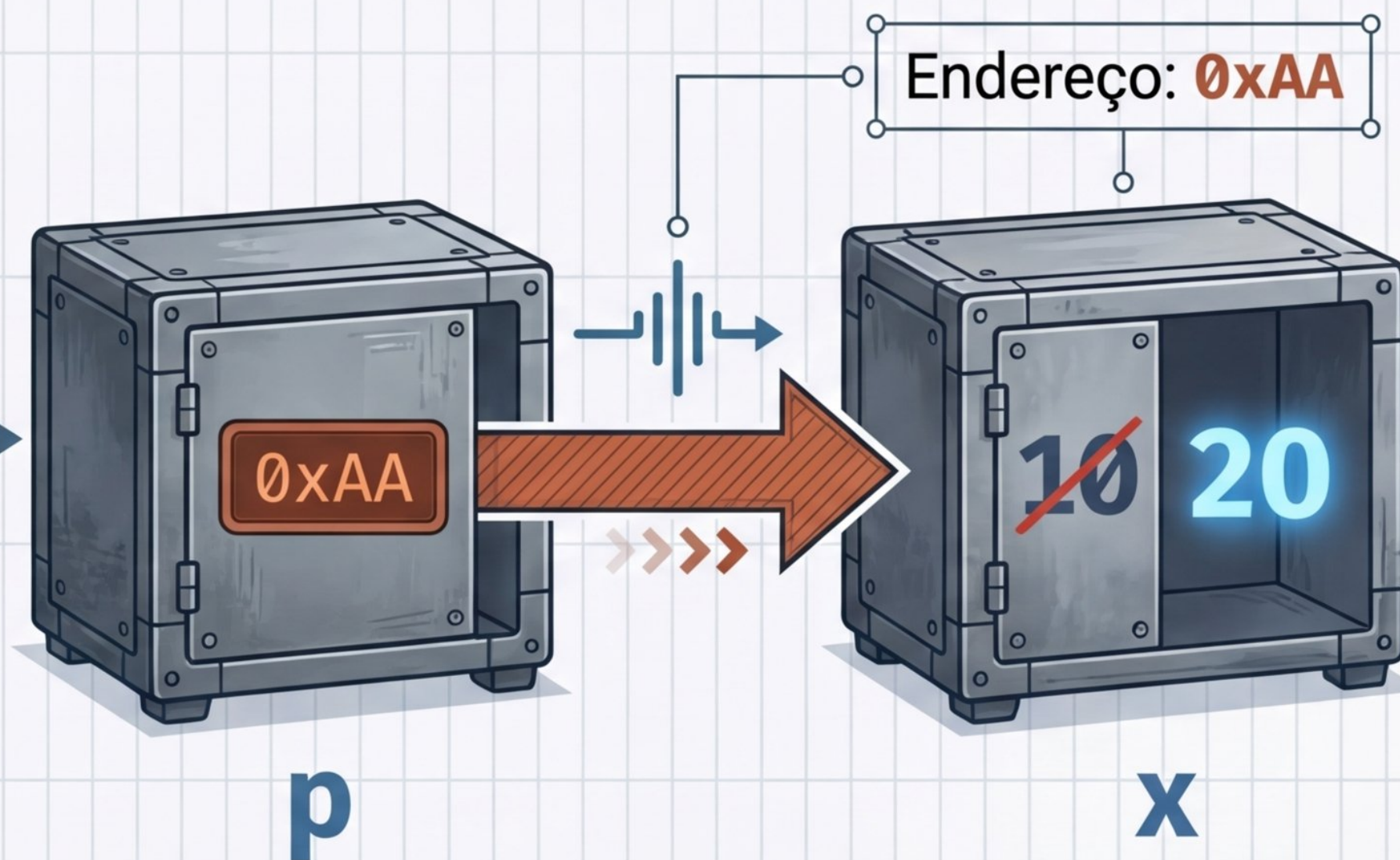
Raio-X da Memória em Execução

Progressão do Código

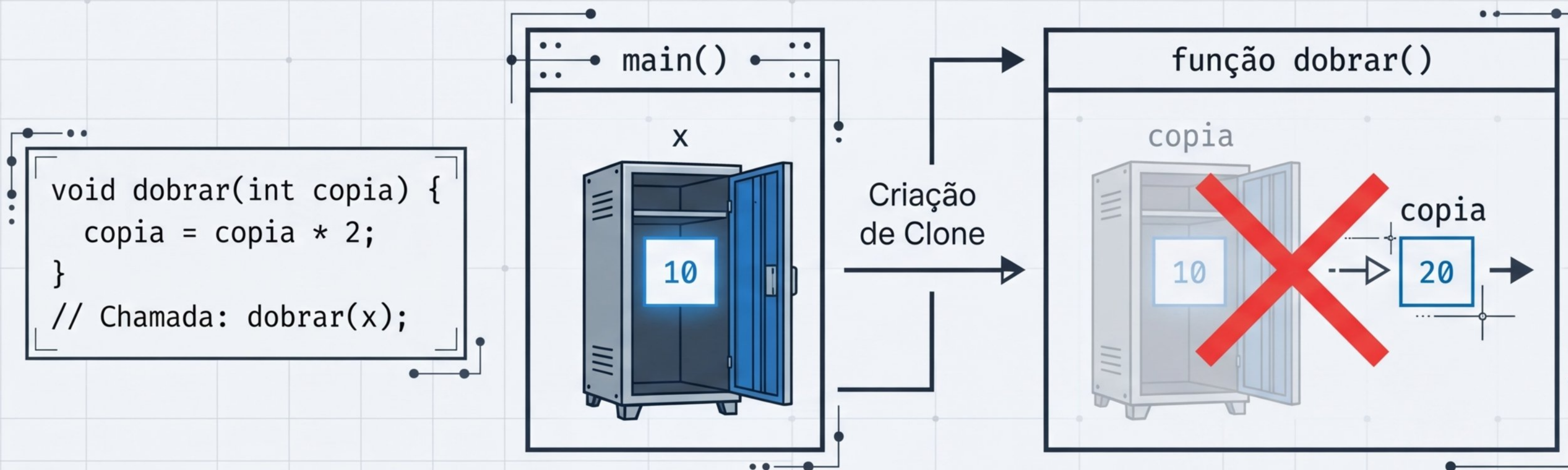
```
int x = 10;
```

```
int *p = &x;
```

```
*p = 20;
```



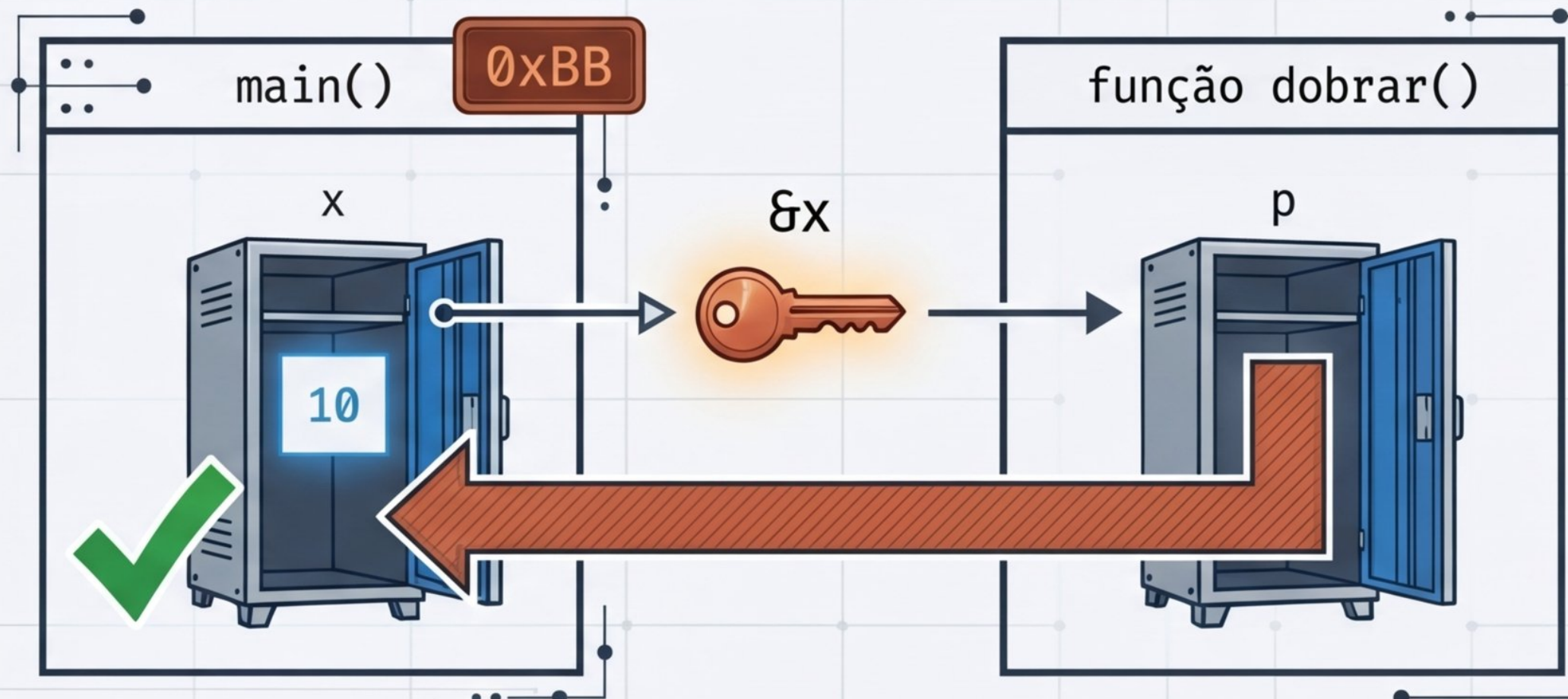
O Problema: A Ilusão da "Passagem por Valor"



Explicação: Quando passamos `x` diretamente para a função, o C cria um clone absoluto. A função `dobrar` altera apenas o clone. A variável original `x` na função principal (`main`) permanece intacta e inalterada.

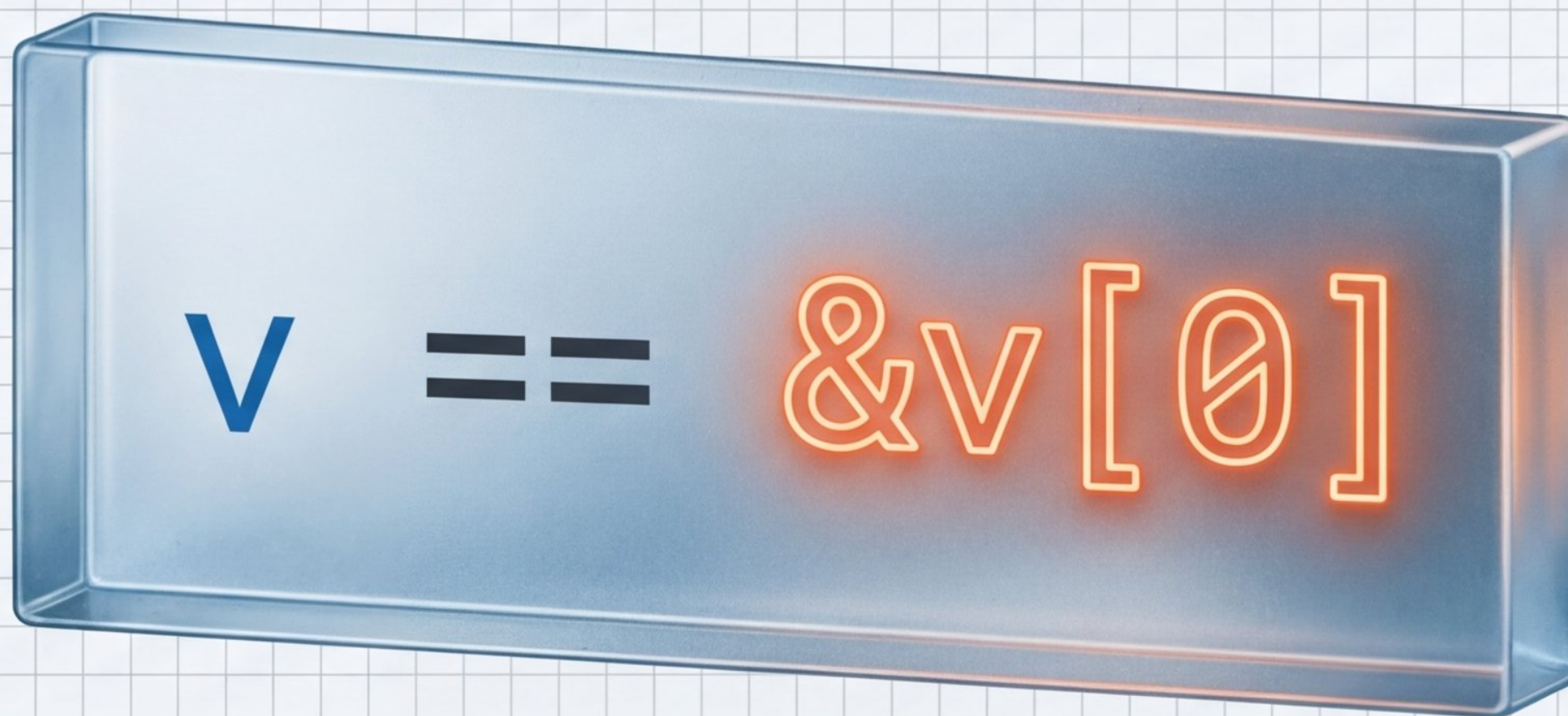
A Solução: Passagem por Referência

```
void dobrar(int *p) {  
    *p = *p * 2;  
}  
// Chamada: dobrar(&x);
```



Explicação: Em vez de enviar uma cópia do dado, enviamos a chave (o endereço com **&x**). A função agora possui um ponteiro **p** que acessa e modifica a variável original diretamente na memória.

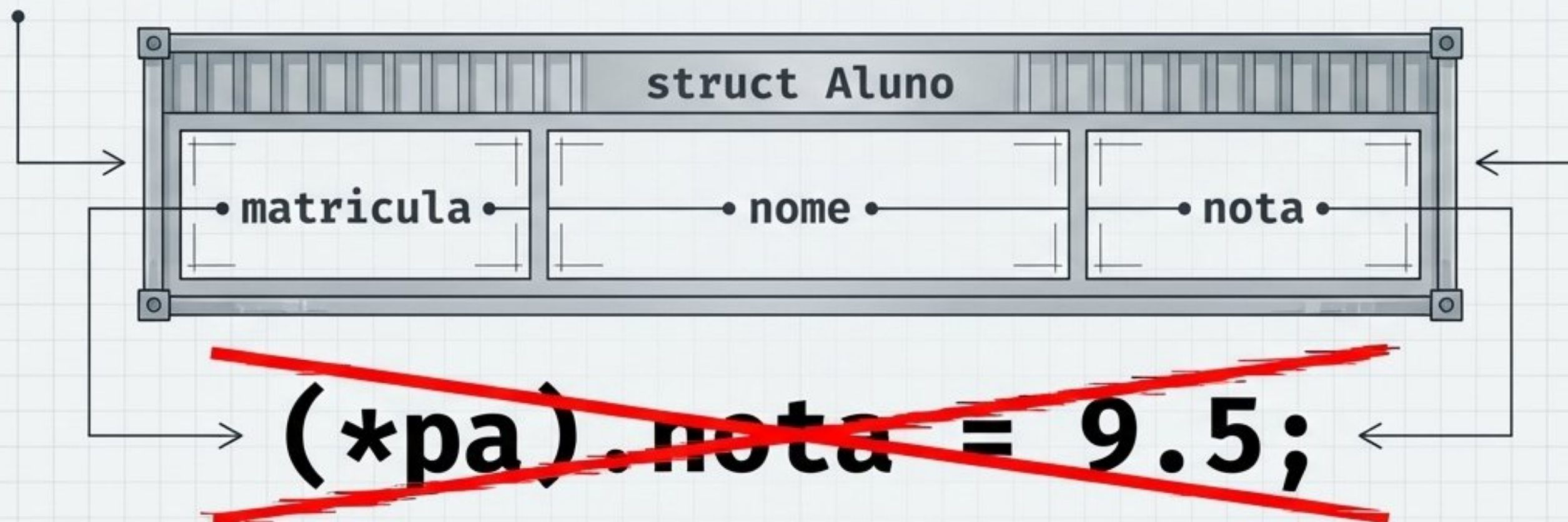
O Segredo dos Vetores: Índices são Ponteiros Disfarçados



Explicação: Em C, o nome de um vetor não é apenas um nome. Ele é, nos bastidores, um ponteiro fixo que aponta exatamente para o endereço de memória do seu primeiro elemento (índice 0).

O Desafio de Apontar para Estruturas (struct)

Contexto: Temos um ponteiro `pa` que aponta para uma `struct Aluno`. Como acessamos a nota desse aluno?



...

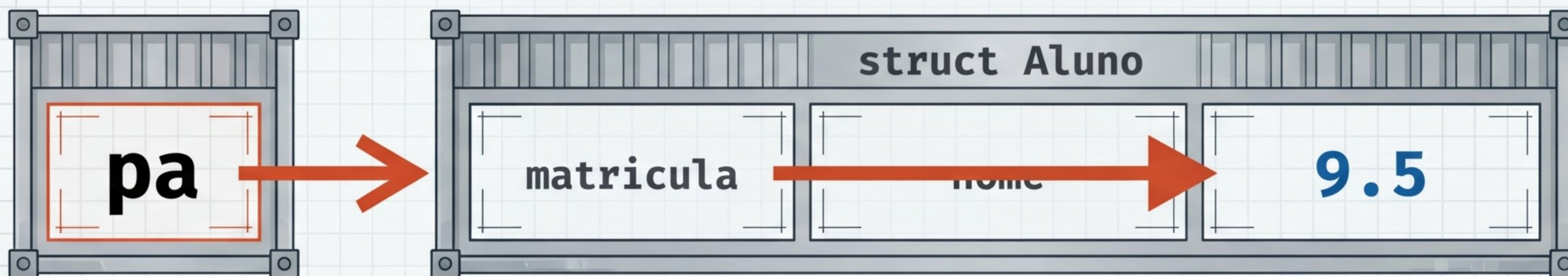
O Problema: A sintaxe exige **parênteses obrigatórios** por causa da ordem de precedência dos operadores em C. Esquecer os parênteses resulta em erros graves de compilação. É **visualmente poluído e propenso a falhas humanas**.

...

A Sintaxe Elegante: O Operador Seta (->)

Contexto: A linguagem C oferece um atalho limpo e direto para acessar campos de uma **struct** através de um ponteiro.

pa->nota = 9.5;



...

- **Explicação:** O operador `->` substitui completamente a necessidade de desreferenciar a struct inteira com `(*pa)`. Ele significa: 'Siga este ponteiro e acesse este campo específico'.

...

Mão na Massa: Laboratório Prático

>_ Missão 1: Mapeamento de Memória

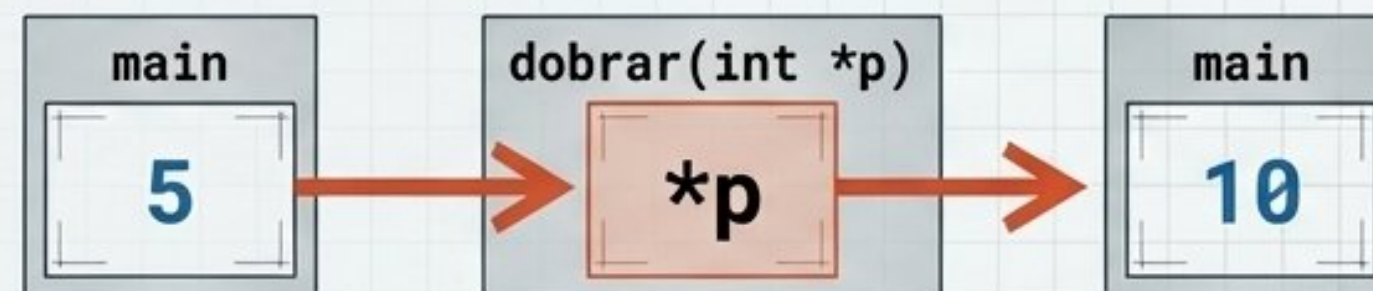


← %p
(e.g., 0x7fff5fbff7c0)

Crie um programa simples que imprima o valor de uma variável e seu endereço de memória lado a lado usando %p no printf.

>_ Missão 2: Alteração Indireta

Implemente a função void dobrar(int *p) e teste a passagem por referência alterando uma variável da main.



>_ Missão 3: Manipulação de Registros

Declare um ponteiro para uma struct Aluno utilizando exclusivamente o operador ->.

Declare um ponteiro para uma struct Aluno. Realize a leitura de dados via teclado e a impressão na tela utilizando exclusivamente



Desafio Final: Atividade Verificadora

Mission Briefing:

- **O Cenário:** Gestão de Preços de Estoque.
Em um sistema de gerenciamento de inventário, a atualização precisa dos preços dos produtos diretamente na memória é crucial para a integridade dos dados.
- **A Estrutura:** Defina uma struct `Produto` contendo `codigo`, `descricao` e `preco`.

```
struct Produto { int codigo; char descricao[50]; float preco; };
```
- **A Missão:** Crie uma função que receba um ponteiro para um produto (`struct Produto *p`) e um percentual de aumento. A função deve atualizar o campo de preço do produto diretamente na memória.

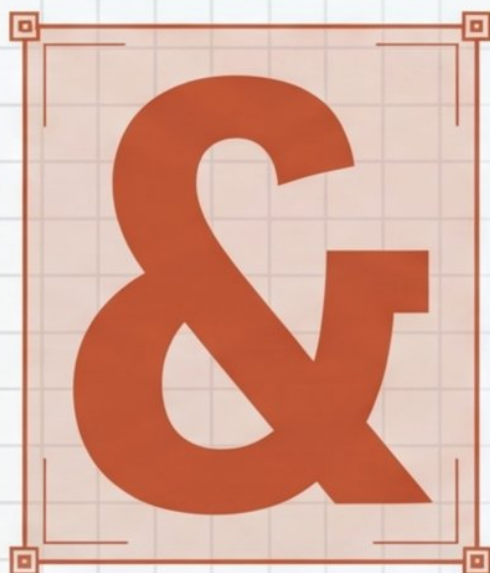


```
void atualizarPreco(struct Produto *p, float percentual)
{
    p->preco = p->preco * (1 + percentual / 100.0);
}
```

- **Critério de Sucesso:** O uso correto do operador de endereço (&) na chamada da função e do operador seta (->) dentro da função para o cálculo.

Síntese da Semana: Guia Rápido de Memória

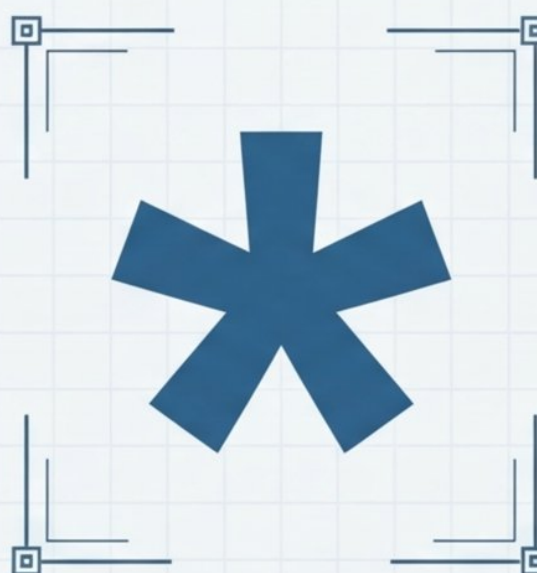
O Endereço (&)



Ação: Extrai a coordenada física.

Uso: `scanf('%d', &x);` ou
`p = &x;`

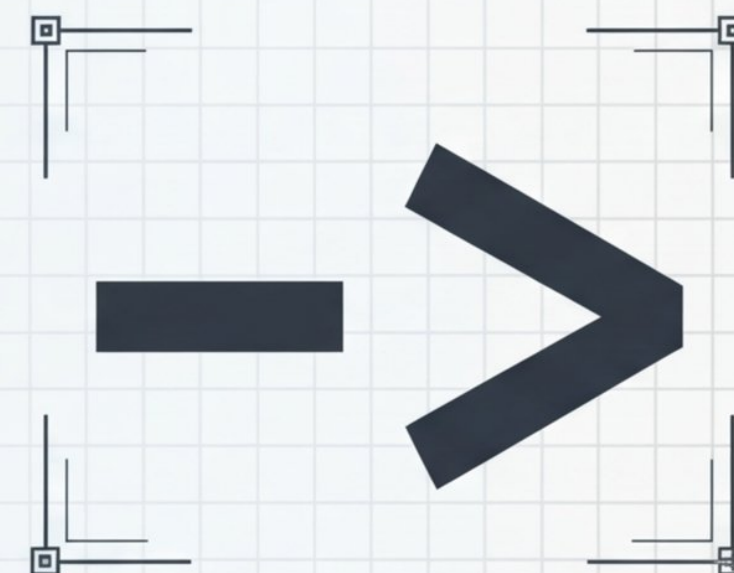
A Indireção (*)



Ação: Destranca o endereço e acessa o valor real.

Uso: `int *p; (Declaração)` |
`*p = 10; (Alteração)`

A Seta de Struct (->)



Ação: Acessa um campo específico de uma struct apontada.

Uso: `pa->preco = 15.99;`

Regra de Ouro: Nunca acesse ou modifique o conteúdo de um ponteiro que não foi inicializado com um endereço válido.